TÌM KIẾM ĐỐI KHÁNG VÀ TRÒ CHƠI

1. Mục Tiêu

- Mô hình hóa trò chơi hai người dưới dạng bài toán tìm kiếm đối kháng.
- Hiểu và cài đặt Minimax, cắt tỉa Alpha-Beta và tìm kiếm Alpha-Beta có hàm đánh giá heuristic.
- Hiểu mối liên hệ giữa AND-OR Search và trò chơi khi hành động của đối thủ được xem như kết quả không xác định.
- Cài đặt và đánh giá Pure Monte Carlo Search và Monte Carlo Tree Search với chính sách chọn UCB1.
- Thực nghiệm, so sánh chất lượng nước đi, số nút duyệt và thời gian chạy của các chiến lược chơi.
- **File tham chiếu:** tictactoe.py; các notebook Tic-Tac-Toe; assignment_connect4.ipynb; assignment_mean_connect4.ipynb; assignment_dots_and_boxes.ipynb.

2. Bài tập thực hành

Bài 1. Định nghĩa trò chơi Tic-Tac-Toe

Mở notebook tictactoe_definitions.ipynb và tập tin tictactoe.py. Xác định cách biểu diễn trạng thái, hành động hợp lệ, mô hình chuyển trạng thái, trạng thái kết thúc và hàm lợi ích.

```
Mô hình trò chơi cần nắm:
• Actions(s): tập các nước đi hợp lệ ở trạng thái s.
• Result(s, a): trạng thái nhận được sau khi thực hiện hành động a.
• Terminal(s): kiểm tra trò chơi đã kết thúc hay chưa.
• Utility(s): giá trị kết quả từ góc nhìn người chơi Max.
• Bàn cờ Tic-Tac-Toe được biểu diễn bằng vector 9 phần tử với các giá trị ' ', 'x', 'o'.
```

Chạy các ví dụ trong notebook, quan sát cách đánh số ô và thử các hàm actions(), result(), terminal(), utility(), check_board() và show_board().

```
board = empty_board()
show_board(board)
print(actions(board))
board = result(board, 'x', 4)
show_board(board)
```

Thực nghiệm cơ sở: cho hai random_player chơi với nhau 1.000 ván bằng hàm play(). Ghi lại số ván X thắng, O thắng và hòa. Nhận xét lợi thế của người đi trước.

Bài 2. Giải Tic-Tac-Toe bằng AND-OR Tree Search

Mở tictactoe_and_or_tree_search.ipynb. Trong mô hình này, hành động của đối thủ được xem như một kết quả không xác định của môi trường. Tìm hiểu các hàm results(), and_or_search(), or_search() và and_search().

Thực hiện các trường hợp kiểm thử có sẵn: X sắp thắng, X cần tìm đường hòa, tình huống bất lợi và bàn cờ rỗng. Với mỗi trường hợp, ghi nước đi được chọn và giải thích kế hoạch điều kiện mà AND-OR Search tạo ra.

Cho and_or_player đấu với random_player. Chạy tối thiểu 100 ván và ghi kết quả.

Bài 3. Minimax và cắt tỉa Alpha-Beta

Mở tictactoe_alpha_beta_tree_search.ipynb. So sánh hai phiên bản: Minimax đệ quy và Minimax có Alpha-Beta Pruning.

```
minimax_search(board, player='x')
alpha_beta_search(board, player='x')
```

Yêu cầu: chạy cùng một tập trạng thái kiểm thử cho cả hai thuật toán; ghi nước đi, giá trị Minimax, số nút đã xét và thời gian chạy (nếu notebook cung cấp). Kiểm tra rằng cắt tỉa Alpha-Beta không thay đổi quyết định tối ưu.

Thử phần Move ordering. So sánh số nút duyệt trước và sau khi thay đổi thứ tự nước đi. Giải thích vì sao thứ tự nước đi tốt có thể tăng hiệu quả cắt tỉa.

Bài 4. Alpha-Beta với hàm đánh giá heuristic và giới hạn độ sâu

Mở tictactoe_heuristic_alpha_beta_tree_search.ipynb. Tìm hiểu hàm eval_fun(state, player), tham số cutoff và hai tác tử heuristic2_player, heuristic4_player.

Chạy các trạng thái kiểm thử giống Bài 3. Thay đổi độ sâu cắt (ví dụ 2 và 4), sau đó so sánh chất lượng nước đi, số nút duyệt và thời gian. Nhận xét sự đánh đổi giữa độ sâu tìm kiếm và chất lượng quyết định.

Phương pháp	Độ sâu/cấu hình	Nước đi	Số nút	Thời gian	Nhận xét
Alpha-Beta đầy đủ	-				
Heuristic Alpha-Beta	cutoff = 2				
Heuristic Alpha-Beta	cutoff = 4				

Bài 5. Pure Monte Carlo Search

Mở tictactoe_pure_monte_carlo_search.ipynb. Tìm hiểu playout(), playouts() và pmcs(). Thuật toán đánh giá từng nước đi bằng nhiều ván mô phỏng ngẫu nhiên thay vì sử dụng hàm heuristic thủ công.

Thử pmcs với N = 10, 100 và 1000 trên cùng các trạng thái kiểm thử. Quan sát độ ổn định của nước đi và thời gian chạy. Sau đó cho pmcs10_player và pmcs100_player đấu với random_player.

Số playout N	Tỷ lệ thắng	Tỷ lệ hòa	Tỷ lệ thua	Thời gian	Nhận xét
10					
100					
1000					

Bài 6. Monte Carlo Tree Search với UCB1

Mở tictactoe_monte_carlo_tree_search_restricted.ipynb. Notebook áp dụng Upper Confidence Bound vào cây ở độ sâu 1 thông qua hàm UCT_depth1(). So sánh với Pure Monte Carlo Search.

Chạy ucb1_10_player và ucb1_100_player trên các trạng thái kiểm thử và khi đấu với random_player. Trả lời: UCB1 cân bằng exploration và exploitation như thế nào? Khi số lần mô phỏng tăng, quyết định có ổn định hơn không?

Bài 7. Chơi Tic-Tac-Toe tương tác

Mở tictactoe_interactive.ipynb. Chạy interactive_player() và thử chơi với một tác tử đã xây dựng ở các phần trên. Thử cả vai X và O. Ghi lại ít nhất một ván đấu và nhận xét chiến lược của tác tử.

3. Bài tập mở rộng

Bài 8. Connect 4

Mở assignment_connect4.ipynb. Hoàn thành lần lượt các nhóm nhiệm vụ được notebook yêu cầu: xác định kích thước không gian trạng thái/cây trò chơi; xây dựng môi trường và Random Agent; Minimax + Alpha-Beta; move ordering; heuristic Alpha-Beta với cutoff; Pure Monte Carlo Search.

Bắt đầu với bàn nhỏ để kiểm thử, sau đó tăng dần kích thước tới bàn chuẩn 6 × 7. Ghi nhận thời gian chọn nước đi và giải thích vì sao Minimax đầy đủ không khả thi trên bàn chuẩn.

Bài 9. Mean Connect 4

Mở assignment_mean_connect4.ipynb. Thực hiện quy trình tương tự Connect 4, nhưng lưu ý luật và hàm utility của biến thể "Mean" Connect 4. So sánh hành vi tác tử giữa hai trò chơi.

Bài 10. Dots and Boxes

Mở assignment_dots_and_boxes.ipynb. Xây dựng biểu diễn bàn cờ và hiển thị, cài đặt result(s,a), utility(s), terminal(s), actions(s), Random Agent, Alpha-Beta và Heuristic Alpha-Beta. Thử tăng kích thước bàn và xác định kích thước lớn nhất có thể xử lý trong thời gian hợp lý.

4. Tổng hợp và báo cáo kết quả

Lập bảng tổng hợp cho các chiến lược Tic-Tac-Toe trên cùng điều kiện thí nghiệm. Mỗi cặp tác tử nên chạy tối thiểu 100 ván; với các thí nghiệm nhẹ có thể dùng 1.000 ván như notebook gợi ý.

Tác tử/Thuật toán	Đối thủ	Số ván	Thắng	Hòa	Thua	Thời gian TB/nước đi
Random	Random					
AND-OR	Random					
Alpha-Beta	Random					
Heuristic Alpha-Beta	Random					
Pure Monte Carlo	Random					
UCB1/MCTS	Random					

Câu hỏi thảo luận:

- Minimax khác AND-OR Search ở cách mô hình hóa đối thủ như thế nào?
- Alpha-Beta có làm thay đổi nước đi tối ưu của Minimax không? Vì sao?
- Khi nào cần dùng heuristic và cutoff thay vì tìm kiếm tới trạng thái kết thúc?
- Pure Monte Carlo Search có ưu điểm gì so với heuristic search? Hạn chế chính là gì?
- UCB1 cải thiện việc phân bổ số lần mô phỏng so với chia đều playout cho các nước đi như thế nào?
- Trong Connect 4 và Dots and Boxes, yếu tố nào làm không gian tìm kiếm tăng nhanh nhất?

5. Yêu cầu nộp bài

- Notebook đã chạy đầy đủ các phần được yêu cầu, có kết quả đầu ra và nhận xét ngay dưới mỗi thí nghiệm.
- Bảng tổng hợp kết quả so sánh các thuật toán và tối thiểu 3–5 nhận xét về chất lượng nước đi, số nút, thời gian và khả năng mở rộng.
- Mã nguồn bổ sung (nếu có) và README ngắn hướng dẫn cách chạy.
- Đối với phần mở rộng, nộp tối thiểu một trong ba bài: Connect 4, Mean Connect 4 hoặc Dots and Boxes theo yêu cầu của giảng viên.

---o0o---

(Hết)